

Socialize Backend

Database Design

Entity-relationship layout for the Postgres schema behind the Socialize training API — every table, key, and constraint that Clean Architecture’s `Socialize.Infrastructure` layer maps onto.

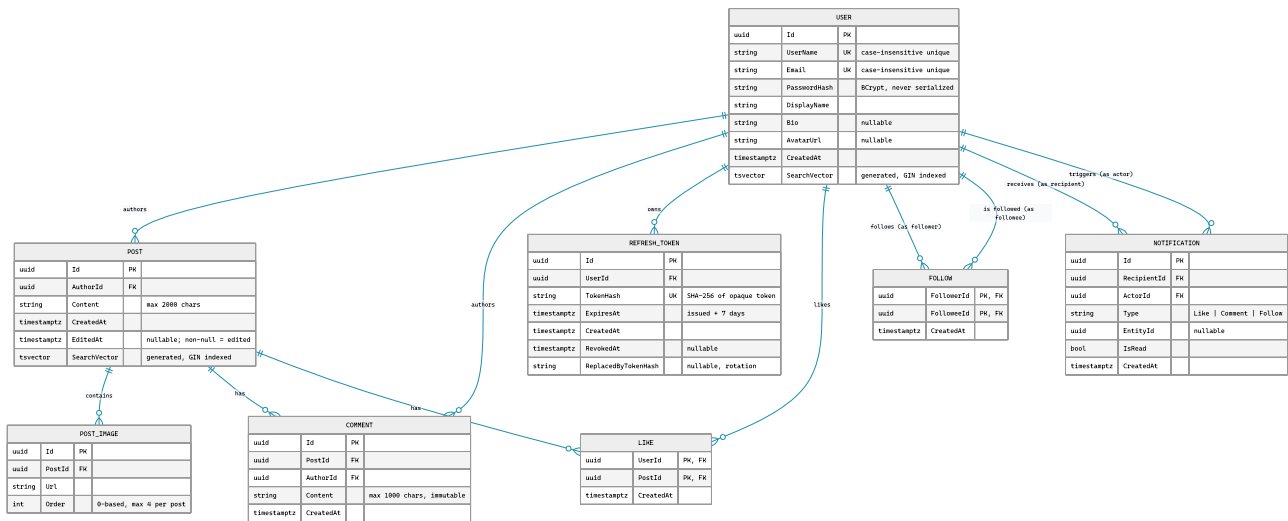
ENGINE	ENTITIES	RELATIONS	ORM
PostgreSQL 16	08	11	EF Core 8

- PK** Primary key
- FK** Foreign key
- UK** Unique constraint
- PK, FK** Composite key member
- CASCADE** deletes children
- RESTRICT** blocks parent delete

01

ENTITY-RELATIONSHIP DIAGRAM

rendered with Mermaid — scroll horizontally on small screens



Composite-key tables (Like, Follow) have no surrogate Id — the pair itself is the key. Follow and Notification each carry two independent foreign keys into User.

02 ENTITY FIELD REFERENCE

full column detail behind the diagram above

User8 COLUMNS			RefreshToken7 COLUMNS		
Id PK	uuid	Surrogate key	Id PK	uuid	Surrogate key
UserName UK	varchar(30)	Case-insensitive unique, 3-30 chars	UserId FK	uuid	→ User.Id, cascade on delete
Email UK	varchar(256)	Case-insensitive unique	TokenHash UK	text	SHA-256 of the opaque token
PasswordHash	text	BCrypt hash; never serialized to clients	ExpiresAt	timestampz	Issued + 7 days
DisplayName	varchar(50)	Required	CreatedAt	timestampz	
Bio	varchar(300)	Nullable	RevokedAt	timestampz	Nullable; set on rotation or logout
AvatarUrl	text	Nullable relative URL	ReplacedByTokenHash	text	Nullable audit chain link
CreatedAt	timestampz	Default now()			
SearchVector	tsvector	Generated (UserName + DisplayName), GIN index			

Post6 COLUMNS			PostImage4 COLUMNS		
Id PK	uuid	Surrogate key	Id PK	uuid	Surrogate key
AuthorId FK	uuid	→ User.Id, restrict on delete	PostId FK	uuid	→ Post.Id, cascade on delete
Content	varchar(2000)	Required	Url	text	Relative path under /uploads/posts
CreatedAt	timestampz	Indexed with Id for	Order	int	0-based position, max 4 per post

		keyset paging
EditedAt	timestampz	Nullable; non-null flags "edited"
SearchVector	tsvector	Generated (Content), GIN index

Comment		5 COLUMNS
Id PK	uuid	Surrogate key
PostId FK	uuid	→ Post.Id, cascade on delete
AuthorId FK	uuid	→ User.Id, restrict on delete
Content	varchar(1000)	Immutable once posted — no edit endpoint
CreatedAt	timestampz	Indexed with PostId + Id for paging

Like		COMPOSITE KEY
UserId PK, FK	uuid	→ User.Id, restrict on delete
PostId PK, FK	uuid	→ Post.Id, cascade on delete
CreatedAt	timestampz	

Follow		COMPOSITE KEY
FollowerId PK, FK	uuid	→ User.Id, restrict on delete
FolloweeId PK, FK	uuid	→ User.Id, restrict on delete
CreatedAt	timestampz	Drives home-feed composition

Notification		6 COLUMNS
Id PK	uuid	Surrogate key
RecipientId FK	uuid	→ User.Id, restrict on delete
ActorId FK	uuid	→ User.Id, restrict on delete
Type	enum	Like · Comment · Follow
EntityId	uuid	Nullable; PostId for Like/Comment

IsRead

bool

Default false

03

DELETE BEHAVIOR

not encoded in the diagram above — captured here instead

Mermaid's ER notation doesn't express **ON DELETE** rules, so every foreign key's behavior is spelled out below. The pattern: children of a Post cascade with it; anything touching User restricts, since there's no delete-account endpoint and several tables carry two independent foreign keys back to User.

RELATIONSHIP	ON DELETE OF PARENT	RATIONALE
Post → PostImage	CASCADE	Images have no meaning without their post
Post → Comment	CASCADE	Comments have no meaning without their post
Post → Like	CASCADE	Likes have no meaning without their post
User → Post (<i>author</i>)	RESTRICT	No delete-account endpoint exists; avoids surprise mass-deletes
User → Comment (<i>author</i>)	RESTRICT	Same reasoning; also dodges EF Core's multiple-cascade-path error via Post
User → RefreshToken	CASCADE	Tokens are meaningless without their owning user
User → Like (<i>liker</i>)	RESTRICT	Avoids a second cascade path into Like alongside Post → Like
User → Follow (<i>either side</i>)	RESTRICT	Avoids a cascade path colliding with the composite key's dual FK
User → Notification (<i>either side</i>)	RESTRICT	Same reasoning as Follow

